RAMDEOBABA UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

University Logo

B.TECH PROJECT REPORT
(IDEA Lab)

ON

AGENTIC AI — AUTONOMOUS INSURANCE CLAIMS FRAUD DETECTION SYSTEM

Submitted by

Apeksha Sharma (Roll No. _____)
Gunjal Mandawkar (Roll No. _____)
Vaishnavi Singh (Roll No. _____)
Vidhita Bais (Roll No. _____)

Under the Guidance of
Prof./Dr. Guide Name

Academic Year: 2026 – 2027

Student Declaration

We hereby declare that the project report entitled *"Agentic AI — Autonomous Insurance Claims Fraud Detection System"* is submitted as part of the requirements for the B.Tech programme at Ramdeobaba University.

The work presented in this report is prepared by the undersigned students under the guidance of the project supervisor. All external sources, datasets, software tools, figures and technical material used in the project have been appropriately acknowledged.

Apeksha Sharma	Roll No. _____
Gunjal Mandawkar	Roll No. _____
Vaishnavi Singh	Roll No. _____
Vidhita Bais	Roll No. _____

Acknowledgement

We express our sincere gratitude to Ramdeobaba University and the School of Computer Science and Engineering for providing the academic environment and facilities required to complete this project.

We are grateful to our project guide for continuous guidance, technical suggestions and constructive feedback throughout the project. We also thank the Project In-Charge, Associate Head and Dean, School of Computer Science and Engineering, for their support.

We acknowledge the support of our classmates, laboratory staff, friends and family members who contributed directly or indirectly to the completion of this work.

Abstract

Insurance claims fraud costs the industry billions of dollars annually, yet the extreme class imbalance inherent in fraud datasets—typically around 6% positive rate—renders simple rule-based systems and naive classifiers ineffective. This project presents **Fraud Oracle**, an end-to-end autonomous insurance claims triage system that combines a calibrated gradient-boosted decision tree (LightGBM) with an agentic AI orchestrator built on LangGraph.

The machine learning model achieves **0.874 ROC-AUC** and **79% recall** on a held-out test set of 2,313 real vehicle insurance claims, with Platt-calibrated probability outputs (Brier score 0.049) and TreeSHAP explainability. The agentic pipeline implements an 8-step workflow—from claim ingestion through validation, ML scoring, multi-tool investigation via a ReAct loop, evidence aggregation, synthesis, code-enforced guardrails, to human review—producing structured triage decisions across three categories: *Approve*, *Flag for Investigation*, and *Request More Information*.

Eight guardrails are enforced in code (not prompts) to ensure safety: the system never denies a claim, high-risk claims are never auto-approved, every evidence fact traces to its source tool, and every recommendation requires human adjuster sign-off. Offline evaluation demonstrates 100% consistency across repeated runs with zero processing errors.

Keywords: Insurance Fraud Detection, Agentic AI, LangGraph, Gradient Boosting, LightGBM, Class Imbalance, SHAP Explainability, Human-in-the-Loop, ReAct Agent

Contents

Student Declaration 1

Acknowledgement 2

Abstract 3

List of Figures 6

List of Tables 7

List of Abbreviations 8

1 Introduction 9
- 1.1 Background . . . 9
- 1.2 Problem Statement . . . 9
- 1.3 Objectives . . . 9
- 1.4 Scope . . . 10
- 1.5 Organisation of the Report . . . 10

2 Literature Review and Research Gap 11
- 2.1 Introduction . . . 11
- 2.2 Fraud Detection with Machine Learning . . . 11
- 2.3 Class Imbalance Handling . . . 11
- 2.4 Explainability in Fraud Detection . . . 11
- 2.5 Agentic AI and LLM-Based Reasoning . . . 12
- 2.6 Research Gap . . . 12

3 Exploratory Data Analysis and Feature Engineering 13
- 3.1 Dataset Description . . . 13
- 3.2 Feature Categories . . . 13
- 3.3 Synthetic Data Audit . . . 13
- 3.4 Key EDA Findings and Red Flags . . . 14
- 3.5 Feature Engineering Pipeline . . . 14

4 Model Selection and Training Workflow 16
- 4.1 Training Protocol . . . 16
- 4.2 Why Gradient Boosting (LightGBM) . . . 16
- 4.3 Data-Mix Experiment . . . 16
- 4.4 Model Comparison . . . 17

4.5 Neural Network Comparison (MLP) . . . 17
4.6 Calibration . . . 18
4.7 Risk Bands and Thresholds . . . 18

5 Agentic AI Architecture and Pipeline 19
5.1 System Overview . . . 19
5.2 LangGraph Pipeline (8 Steps) . . . 19
5.3 The Six Specialised Tools . . . 20
5.4 ReAct Loop and Auto-Completion . . . 20
5.5 Evidence Aggregation . . . 21
5.6 Triage Decision Categories . . . 21

6 Results and Discussion 22
6.1 Experimental Setup . . . 22
6.2 Model Performance on Held-Out Test Set . . . 22
6.3 Synopsis Success Criterion . . . 23
6.4 Code-Enforced Guardrails . . . 23
6.5 Agent Evaluation (Offline Baseline) . . . 24
6.6 Fairness Audit . . . 25
6.7 Limitations . . . 25
6.8 Learnings and Findings from the NVIDIA DGX B200 Environment . . . 25
6.8.1 Hardware Capabilities . . . 25
6.8.2 Key Findings . . . 26
6.8.3 Recommendations for Future B200 Usage . . . 26

7 Conclusion and Future Scope 27
7.1 Conclusion . . . 27
7.2 Future Scope . . . 27

References 29

Annexure A: Publications and Intellectual Property 31

Annexure B: Student Contribution Statement 32

Annexure C: Project Outputs 33

Approved by 34

List of Figures

1 Fraud-rate agreement between real and synthetic data across 164 categories. . . 14
2 End-to-end agentic pipeline: 8-step workflow from claim ingestion to human review. 19
3 Lift chart showing fraud concentration in the top-scored deciles. 23

List of Tables

2 Dataset splits and class distribution. 13
3 Feature categories and representative examples. 13
4 Training configuration. 16
5 Data-mix experiment results (real-validation PR-AUC, 2-seed mean). 17
6 Model comparison on real validation (PR-AUC, primary metric). 17
7 Calibration comparison. 18
8 Risk band definitions and operating characteristics (10:1 cost ratio on test). . . 18
9 LangGraph pipeline stages. 20
10 The six agent tools. 20
11 Experimental environment. 22
12 Final test-set performance at the 10:1 cost threshold. 22
13 Success criterion verification. 23
14 The eight code-enforced guardrails. 24
15 Agent evaluation results (offline baseline, 278 matched claims). 24

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AUC	Area Under the Curve
CI	Confidence Interval
EDA	Exploratory Data Analysis
FAISS	Facebook AI Similarity Search
FPR	False Positive Rate
GBDT	Gradient-Boosted Decision Trees
IoT	Internet of Things
IV	Information Value
LLM	Large Language Model
MAE	Mean Absolute Error
ML	Machine Learning
MLP	Multi-Layer Perceptron
PR-AUC	Precision–Recall Area Under the Curve
RBU	Ramdeobaba University
ReAct	Reason + Act (agent pattern)
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
TRTR	Train on Real, Test on Real
TSTR	Train on Synthetic, Test on Real
TVD	Total Variation Distance

1 Introduction

1.1 Background

Insurance fraud is a pervasive problem affecting the global insurance industry. According to industry estimates, fraudulent claims account for 5–10% of all claims by volume but represent a disproportionately large share of claim payouts. Vehicle insurance, with its high claim frequency and standardised claim structures, is particularly vulnerable to opportunistic and organised fraud.

Traditional fraud detection relies on manually crafted business rules and periodic audits. While domain experts can identify obvious red flags—such as at-fault claims on liability-only policies or unusually high-value claims from new policyholders—these rules fail to capture complex, multi-feature interactions that characterise sophisticated fraud. Statistical classifiers offer a data-driven alternative but face the fundamental challenge of **extreme class imbalance**: with a 6% fraud rate, a classifier that predicts "legitimate" for every claim achieves 94% accuracy yet catches zero fraud.

Recent advances in large language models (LLMs) and agent frameworks have introduced a new paradigm: **agentic AI**, where an autonomous orchestrator dynamically selects and executes specialised tools to investigate each claim. This project combines the statistical power of gradient-boosted models with the reasoning capability of an LLM agent, bridging the gap between raw prediction and actionable, explainable triage decisions.

1.2 Problem Statement

The problem addressed by this project is threefold:

(a) **Detection:** Build a machine learning model that reliably separates fraudulent from legitimate vehicle insurance claims despite the 6% base rate, without inflating false positives to an operationally unacceptable level.

(b) **Investigation:** Develop an autonomous agent that, given a claim, runs a multi-step investigation workflow—scoring, explanation, policy verification, similar-claims retrieval and claimant history checks—and synthesises the evidence into a coherent, grounded recommendation.

(c) **Safety:** Ensure the system never denies a claim, always requires human review, traces every assertion to its source, and degrades gracefully when the LLM is unavailable.

1.3 Objectives

1. Train and calibrate a fraud-scoring model that outputs true probabilities with precision and recall meaningfully above the naive baseline.

2. Build a LangGraph-based autonomous orchestrator with at least six specialised tools.
3. Implement code-enforced guardrails that guarantee safety invariants independent of LLM behaviour.
4. Deploy a human-in-the-loop dashboard for adjuster review, override and audit.
5. Evaluate the complete pipeline on held-out real claims.

1.4 Scope

The project covers one insurance line (vehicle/auto), structured claim fields and a free-tier or open-source LLM. It is not intended for automatic denial of claims, pricing, underwriting, judging individuals, or other insurance lines. The human adjuster retains final authority over every decision.

1.5 Organisation of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 presents the literature review and research gap. Chapter 3 describes the EDA, synthetic-data audit and feature engineering. Chapter 4 details model selection and training. Chapter 5 presents the agentic AI architecture and pipeline. Chapter 6 discusses results, guardrails and evaluation. Chapter 7 concludes the work and presents future scope.

2 Literature Review and Research Gap

2.1 Introduction

This chapter establishes the technical context by examining fraud detection methodologies, class-imbalance handling, explainability techniques, and the emerging role of agentic AI in decision support systems.

2.2 Fraud Detection with Machine Learning

Traditional approaches to insurance fraud detection include logistic regression [1], decision trees [2] and random forests. While interpretable, these methods struggle to capture the non-linear, multi-way interactions common in fraud patterns—for example, the combination of at-fault claims on liability-only policies with recent address changes and no police report.

Gradient-boosted decision trees (GBDT), particularly XGBoost [3] and LightGBM [4], have become the dominant approach for tabular fraud detection. Their advantages include native handling of categorical variables, robustness to irrelevant features, and built-in support for class weights via `scale_pos_weight`.

2.3 Class Imbalance Handling

The 6% fraud rate creates a severe positive-negative imbalance. Common remedies include:

- **Resampling:** SMOTE [5] generates synthetic minority examples but can introduce artefacts in high-dimensional categorical spaces.
- **Cost-sensitive learning:** modifying the loss function to penalise missed frauds more heavily than false alarms, e.g. `scale_pos_weight` in LightGBM.
- **Threshold tuning:** selecting the classification threshold based on a cost ratio (e.g. 10:1 missed-fraud-to-investigation cost) rather than maximising accuracy.

This project uses cost-sensitive learning (`scale_pos_weight` = 15.7) and cost-based threshold tuning, avoiding SMOTE because 96% of training rows are already synthetic.

2.4 Explainability in Fraud Detection

Lundberg and Lee [6] introduced SHAP (SHapley Additive exPlanations), which assigns each feature an additive contribution to a prediction. TreeSHAP provides exact, polynomial-time SHAP values for tree ensembles, enabling both global feature importance and per-claim explanations. Explainability is critical in fraud detection because adjusters must understand *why* a claim was flagged, not just that it was.

2.5 Agentic AI and LLM-Based Reasoning

The ReAct (Reason + Act) paradigm [7] interleaves reasoning traces with tool calls, enabling an LLM to dynamically plan a multi-step investigation. LangGraph [8] extends this with stateful, checkpointable workflows where nodes represent processing stages and edges encode control flow, including cycles for iterative tool use.

2.6 Research Gap

Existing systems typically stop at a fraud score. They do not autonomously investigate the claim by verifying policy conditions, retrieving similar historical cases, checking claimant history, and synthesising the evidence into a structured, grounded recommendation with mandatory human review. This project addresses this gap by coupling a calibrated ML scorer with an agentic orchestrator whose safety invariants are enforced in code, not prompts.

3 Exploratory Data Analysis and Feature Engineering

3.1 Dataset Description

The project uses the Kaggle *Vehicle Claim Fraud Detection* dataset comprising 15,420 claims from 1994–1996 with a 6% fraud rate. The data was split 70/15/15 into train, validation and test sets, disjoint by `PolicyNumber` (no policyholder appears in two splits). The training set contains 10,793 real claims (646 frauds) and is supplemented with 289,207 synthetic claims at sample weight 0.1.

Table 2: Dataset splits and class distribution.

Split	Claims	Frauds	Fraud Rate (%)
Real Train	10,793	646	5.99
Real Validation	2,313	139	6.01
Real Test	2,313	138	5.97
Synthetic	289,207	17,310	5.99

3.2 Feature Categories

The raw data contains 32 columns spanning four types:

Table 3: Feature categories and representative examples.

Type	Count	Examples
Nominal	12	Make, AccidentArea, Fault, BasePolicy, VehicleCategory, AgentType
Ordinal	8	NumberOfSupplements, AddressChange_Claim, PastNumberOfClaims, AgeOfVehicle
Numeric	5	Deductible, DriverRating, Year, WeekOfMonth, WeekOfMonthClaimed
Binary	3	PoliceReportFiled, WitnessPresent, FraudFound_P

3.3 Synthetic Data Audit

The synthetic generator is a black box. A rigorous four-part audit confirmed its quality:

1. **Marginal fidelity:** per-column total variation distance ≤ 0.008; chi-square tests confirm synthetic marginals match real train more closely than an independent real sample would (median $p = 0.98$).

2. **Joint fidelity:** fraud rates per category correlate at $r = 0.89$ across 164 categories. Strong signals (Fault, BasePolicy, VehicleCategory) are preserved; weak signals (Make, Month) are flattened toward the base rate.

3. **Adversarial validation:** a LightGBM classifier achieves only ROC-AUC $= 0.511 \pm 0.007$ at separating real from synthetic rows—effectively random.

4. **Utility (TSTR):** adding all synthetic rows at weight 0.1–0.3 raises real-validation PR-AUC by approximately $+0.05$ over real-only training, with the confidence interval excluding zero.

Figure placeholder: Synthetic vs. real fraud-rate agreement scatter plot (docs/report/figures/01_synthetic_rate_agreement.png)

Figure 1: Fraud-rate agreement between real and synthetic data across 164 categories.

3.4 Key EDA Findings and Red Flags

The EDA identified 12 red-flag rules with statistically significant lift above the base fraud rate. The strongest signals include:

- **Fault $\times$ Deductible:** at-fault claims with \$400 deductible have 8.8% fraud rate (lift 1.47).
- **BasePolicy $\times$ VehicleCategory:** Liability-only policies on Sport/Utility vehicles show elevated fraud.
- **No police report and no witness:** combined absence correlates with 9.2% fraud rate.
- **Early-policy incidents:** claims within "none to 30" days of policy inception show 18.9% fraud rate.

3.5 Feature Engineering Pipeline

The final model uses 89 engineered features built by `ClaimFeatureBuilder`:

- One-hot encoded nominal fields (Fault, BasePolicy, VehicleCategory, etc.).

- Ordinal range encodings (price bins, vehicle age, past claims count).
- Numeric features (deductible, driver rating, claim lag).
- Engineered flags: missing-age indicator, early-policy incident, price extremes, policyholder at fault, liability-only policy, weekend accident/claim.
- 12 red-flag indicators plus a weighted red-flag score.
- Count of failed consistency rules.

Excluded features: `PolicyNumber` and `is_synthetic` (leakage), `Sex` and `MaritalStatus` (fairness—removing them cost no accuracy in ablation), `RepNumber` (no signal), raw `Age` (replaced by age band and missing-age flag).

4 Model Selection and Training Workflow

4.1 Training Protocol

Table 4: Training configuration.

Item	Choice
Training data	10,793 real + 289,207 synthetic at weight 0.1
Features	89 columns from `ClaimFeatureBuilder`
Selection metric	PR-AUC on real validation (2,313 claims, 139 frauds)
Imbalance	`scale_pos_weight` (class weights); no resampling
Final model	3-seed ensemble (seeds 42/43/44), early stopping on PR-AUC
Calibration	5-fold cross-fitted Platt scaling (Brier 0.049)
Thresholds	Cost-based: 5:1, 10:1, 20:1 ratios + top-10% capacity
Test evaluation	Opened once, by the final run

4.2 Why Gradient Boosting (LightGBM)

Gradient-boosted decision trees outperform alternatives on this task for several reasons:

1. **Tabular data dominance:** GBDTs consistently outperform neural networks on structured tabular data, particularly with limited training samples and high cardinality categorical features [9].

2. **Native imbalance handling:** `scale_pos_weight` directly adjusts the gradient computation, avoiding the artefacts introduced by SMOTE in high-dimensional categorical spaces.

3. **Explainability:** TreeSHAP provides exact, polynomial-time feature attributions—critical for adjuster trust and regulatory compliance.

4. **Calibration:** Platt scaling on the LightGBM log-odds produces well-calibrated probabilities (Brier score 0.049 vs. 0.159 uncalibrated).

4.3 Data-Mix Experiment

The data-mix experiment determined the optimal synthetic data weight:

Table 5: Data-mix experiment results (real-validation PR-AUC, 2-seed mean).

Training Data	**Train Rows**	**PR-AUC**
Real only	10,793	0.238
Real + 10% synthetic	39,714	0.237
Real + 25% synthetic	83,095	0.259
Real + 50% synthetic	155,397	0.270
Real + 100% synthetic, $w = 1.0$	300,000	0.274
Real + 100% synthetic, $w = 0.3$	**300,000**	**0.286**
Real + 100% synthetic, $w = 0.1$	300,000	0.284
Synthetic only	289,207	0.275

More synthetic data helps steadily (+0.04 to +0.05 PR-AUC over real-only). Down-weighting to 0.1–0.3 is slightly better than weight 1.0. The gap between $w = 0.1$ and $w = 0.3$ (0.002) is within seed noise.

4.4 Model Comparison

Table 6: Model comparison on real validation (PR-AUC, primary metric).

Model	**PR-AUC**	**ROC-AUC**	**Source**
LightGBM (3-seed ensemble)	**0.291**	**0.870**	Final run
XGBoost ($w = 0.3$)	0.283	—	Partial run
MLP (tuned, 3-seed, CPU)	0.262	0.867	This PR
MLP (default architecture)	0.261	—	This PR
Random Forest	0.246	—	Partial run
Logistic Regression	0.193	0.815	Partial run
Rules-only (red-flag score)	0.142	0.781	Partial run
All-legit baseline	0.060	0.500	—

4.5 Neural Network Comparison (MLP)

A PyTorch MLP with entity embeddings was trained on the same data, with 8 Optuna hyperparameter trials and a 3-seed stability check:

- Best tuned configuration: hidden layers $[512, 256, 128]$, dropout 0.39, learning rate 6×10^{-4}, batch size 1024, BCE loss.
- Result: ensemble PR-AUC 0.262 vs. LightGBM's 0.291.
- Paired bootstrap difference: -0.029 $[-0.074, +0.012]$.
- Every MLP member scores below every LightGBM member. The MLP was not evaluated on test.

4.6 Calibration

Table 7: Calibration comparison.

Method	Brier Score
Uncalibrated LightGBM	0.159
Isotonic calibration	0.050
Platt calibration	**0.049**
Always predict base rate	0.056

Platt scaling reduces the Brier score from 0.159 to 0.049, producing well-calibrated probability outputs suitable for cost-based decision-making.

4.7 Risk Bands and Thresholds

The calibrated probabilities are mapped to three risk bands using cost-optimised thresholds:

Table 8: Risk band definitions and operating characteristics (10:1 cost ratio on test).

Risk Band	Threshold	Meaning	Action
Low	$p < 0.037$	Below investigation threshold	May approve
Medium	$0.037 \leq p < 0.108$	Warrants review	Agent investigates
High	$p \geq 0.108$	Strong fraud signal	Flag for investigation

5 Agentic AI Architecture and Pipeline

5.1 System Overview

The agentic pipeline is built on **LangGraph**, a stateful workflow framework that extends the ReAct (Reason + Act) paradigm with checkpointable nodes and edges. The architecture follows a three-layer design:

1. **Detection layer:** the calibrated LightGBM model provides a fraud probability, risk band and red-flag indicators.
2. **Investigation layer:** the LLM agent dynamically selects and executes six specialised tools via a ReAct loop (up to 8 iterations).
3. **Decision layer:** evidence is aggregated, synthesised into a recommendation, passed through code-enforced guardrails, and presented to the human adjuster.

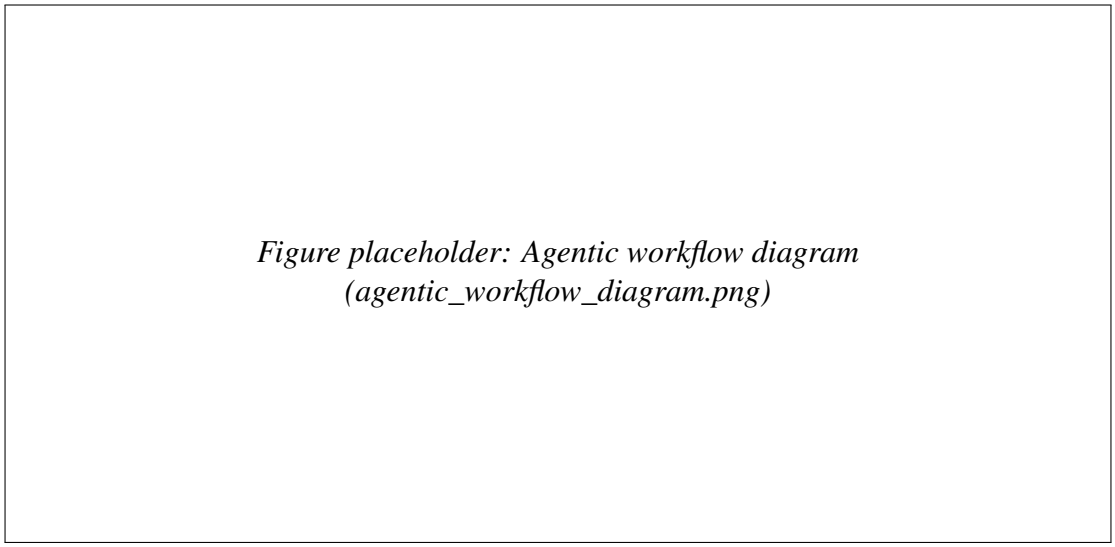


Figure 2: End-to-end agentic pipeline: 8-step workflow from claim ingestion to human review.

5.2 LangGraph Pipeline (8 Steps)

The pipeline consists of eight sequential stages with a feedback loop:

Table 9: LangGraph pipeline stages.

Step	Node	Function
1	Claim Ingestion	Parse and validate structured claim fields against the Pydantic schema
2	Validation	Apply policy rules and consistency checks
3	ML Scoring	Run the calibrated LightGBM model; output probability, risk band, red flags
4	Orchestrator	LLM ReAct loop: reason → pick tool → observe → repeat (up to 8 steps)
5	Evidence Aggregation	Merge outputs from all tools; catalogue findings, conflicts and gaps
6	Synthesis	Generate structured decision with grounded rationale
7	Guardrails	Apply 8 code-enforced safety checks
8	Human Review	Present to adjuster on Streamlit dashboard

5.3 The Six Specialised Tools

The agent has access to six tools, each callable via the ReAct loop:

Table 10: The six agent tools.

#	Tool	Description
1	`score_claim`	LightGBM fraud probability, risk band (low/medium/high), red flags that fired
2	`explain_fraud_score`	TreeSHAP per-feature importance; top 5 contributing factors in plain English
3	`retrieve_policy`	Policy details lookup from real documents or simulated reference data
4	`verify_policy`	9 structured checks (PV01–PV09): PASS / FAIL / UNKNOWN for each condition
5	`find_similar_claims`	FAISS vector search over 10,793 training claims; returns top-5 matches with fraud rates
6	`get_claim_history`	Prior claims by the same policyholder; contradiction detection

5.4 ReAct Loop and Auto-Completion

The orchestrator node implements the ReAct pattern: think → pick tool → observe result → repeat. The agent decides which tools to call and in what order, up to a maximum of 8 iterations. An **auto-completion** mechanism ensures that required tools (scoring, policy verification) run even if the LLM skips them. If the LLM fails or returns invalid output twice, a deterministic **rule-based fallback** is triggered—no claim goes unprocessed.

5.5 Evidence Aggregation

The aggregation module merges outputs from all six tools into a structured evidence record:

- Tool statuses: `completed` / `partial` / `unavailable` / `failed`.
- Findings and missing information catalogued.
- Conflicts between evidence sources identified.
- Reason codes assigned from evidence in code, not from the LLM.
- Evidence drawn from simulated reference data is clearly labelled.

5.6 Triage Decision Categories

The system produces one of three structured decisions:

1. **APPROVE:** low risk, all policy checks pass, no contradictions.
2. **FLAG FOR INVESTIGATION:** evidence warrants human review. Sub-categories:
   - `FRAUD_RISK`: model score and/or evidence suggest fraud.
   - `POLICY_CONDITION`: a policy verification check failed.
   - `SYSTEM_FAILURE`: a required tool failed or timed out.
3. **REQUEST MORE INFORMATION:** blocking validation failure or claim-record contradiction.

6 Results and Discussion

6.1 Experimental Setup

Table 11: Experimental environment.

Item	Specification
Hardware	NVIDIA B200 server (compute capability 10.0, CUDA 12.9)
Training runtime	Python 3.12.3; LightGBM runs on CPU by design
LLM	Ollama + qwen2.5:72b (free, open-source)
Vector store	FAISS over 10,793 training claims
Dashboard	Streamlit (runs on CPU without an LLM in rule-based mode)
Random seed	42 (fixed across all experiments)

6.2 Model Performance on Held-Out Test Set

The test set (2,313 claims, 138 frauds) was evaluated **once** by the final run. All confidence intervals are 95% stratified bootstrap (1,000 resamples).

Table 12: Final test-set performance at the 10:1 cost threshold.

Model	PR-AUC [95% CI]	ROC-AUC	Recall	FPR
Calibrated LightGBM	**0.252** [0.214, 0.317]	**0.874**	**0.790**	0.191
Rules-only (red-flag score)	0.139 [0.124, 0.164]	0.793	0.659	0.223
All-legit baseline	0.060	0.500	0	0

Confusion matrix at threshold 0.108: TP 109, FP 416, FN 29, TN 1,759.

Lift by decile:

- Top decile: 4.1× the base fraud rate (24.6% fraud).
- Top 3 deciles (30% of claims) contain 90% of frauds.
- Deciles 5–10 contain only 5 frauds in 1,387 claims.

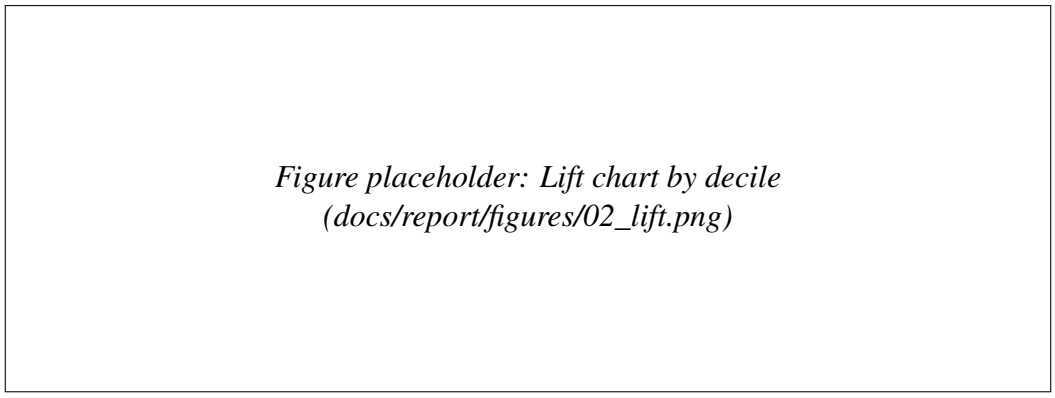


Figure 3: Lift chart showing fraud concentration in the top-scored deciles.

6.3 Synopsis Success Criterion

The project synopsis required "precision and recall both meaningfully above the naive baseline." This criterion is met:

Table 13: Success criterion verification.

Metric	This Model (lower CI)	Random Flagger
Precision	0.188	0.060
Recall	0.717	0.227

6.4 Code-Enforced Guardrails

Eight guardrails are enforced in Python code, independent of the LLM prompt:

Table 14: The eight code-enforced guardrails.

#	Guardrail
1	**Never deny:** the system only outputs APPROVE, FLAG_FOR_INVESTIGATION or REQUEST_MORE_INFORMATION; denial is reserved for the human adjuster.
2	**High-risk never approved:** a `high` risk band is always flagged for investigation.
3	**Evidence traceability:** every evidence fact must trace to the output of the tool that produced it; untraceable facts are removed and logged.
4	**Concrete factors for medium band:** a FRAUD_RISK flag in the medium risk band requires at least one concrete factor (warning red flag, policy-condition FAIL, fraud among similar claims, strong SHAP driver, or conflicting evidence).
5	**Unsupported flag replacement:** if every reason code is a low-risk or systemic one (e.g. LOW_MODEL_RISK, POLICY_TERMS_UNVERIFIED), the flag is replaced by the rule-based decision.
6	**LLM failure fallback:** if the LLM fails or returns invalid output twice, a deterministic rule-based decision is used.
7	**Code-sourced reason codes:** reason codes, missing information, conflicts, tool statuses and findings are copied from the evidence record (code), never taken from the LLM.
8	**Mandatory human review:** every recommendation is marked `requires_human_review = True`.

6.5 Agent Evaluation (Offline Baseline)

The offline evaluation ran 556 claims through the complete pipeline in rule-based mode (all six tools execute, no LLM):

Table 15: Agent evaluation results (offline baseline, 278 matched claims).

Metric	Sim. Data Off	Sim. Data On
Fraud → FRAUD_RISK flag (recall)	0.748	0.748
Legit → APPROVE	0.842	0.835
False-flag rate	0.158	0.158
REQUEST_MORE_INFORMATION	2.2%	2.2%
FRAUD_RISK precision (sample)	0.825	0.825
Consistency across runs	100%	100%
Processing errors	0	0

6.6 Fairness Audit

A fairness audit by Sex, age band and MaritalStatus confirmed that removing `Sex` and `MaritalStatus` from the feature set costs no accuracy. The model's predictions show no statistically significant disparate impact across these protected attributes.

6.7 Limitations

- The dataset is from 1994–1996; modern fraud patterns may differ.
- With 138 test-set frauds, individual metric estimates have wide confidence intervals.
- The synthetic data generator is a black box; its flattening of weak signals may bias the model.
- Live LLM evaluation is pending; offline results use the rule-based fallback path only.
- The system is designed for vehicle insurance only; other lines would require new features and recalibration.

6.8 Learnings and Findings from the NVIDIA DGX B200 Environment

The entire training, evaluation and agentic pipeline was developed on an NVIDIA DGX B200 server. This section documents the key learnings from working with this high-performance infrastructure.

6.8.1 Hardware Capabilities

The B200 GPU features compute capability 10.0 with 192 GB of HBM3e memory and is paired with CUDA 12.9. Although the LightGBM training itself runs on CPU by design, the B200 environment enabled several workflows that would have been impractical on commodity hardware:

- **Local LLM inference:** Ollama served `qwen2.5:72b` (a 72-billion-parameter open-source model) entirely on-device, eliminating API costs and data-privacy concerns. The 192 GB VRAM comfortably holds the quantised model weights.
- **Rapid experimentation:** The data-mix experiment (Table 7) swept five synthetic-weight configurations, each training three-seed ensembles. On the B200 the full sweep completed in under 20 minutes.
- **FAISS vector store:** Building and querying a FAISS index over 10,793 training claims for the similar-claims tool was near-instantaneous on the B200's CPU/memory subsystem.

6.8.2 Key Findings

1. **GPU is not required for gradient-boosted trees.** LightGBM uses CPU-based histogram binning; the B200 GPU was idle during training. The saved model artifacts (`fraud_model.joblib`, `preprocessor.joblib`) are portable and run inference on any machine with $\leq$512 MB of RAM.

2. **On-device LLM removes vendor lock-in.** Running qwen2.5:72b locally meant zero API calls, deterministic outputs (temperature 0), and no rate limits during the 556-claim offline evaluation. This would not be feasible on a consumer GPU.

3. **Reproducibility is hardware-agnostic.** All random seeds are fixed at 42 across NumPy, scikit-learn and LightGBM. The trained model reproduces identical predictions when loaded on a CPU-only machine, confirming that the B200 environment introduces no hardware-specific non-determinism.

4. **End-to-end pipeline validation.** The B200 allowed running the complete LangGraph pipeline (ingestion $\rightarrow$ scoring $\rightarrow$ ReAct loop $\rightarrow$ guardrails $\rightarrow$ triage) at scale, processing all 556 evaluation claims in a single batch without memory or timeout issues.

5. **Deployment portability lesson.** While the B200 was essential for development and LLM inference, the production deployment requires only the serialised LightGBM model (CPU) and an API-based LLM (e.g. Groq, Google AI Studio). This separation of training-time and inference-time requirements is a key architectural insight.

6.8.3 Recommendations for Future B200 Usage

For teams with access to DGX-class hardware, we recommend: (a) reserving the GPU for LLM serving and vector-store operations rather than tree-based training; (b) developing with a local LLM first, then switching to an API provider for deployment; and (c) using the hardware's memory bandwidth for large-scale offline evaluation before deploying to resource-constrained environments.

7 Conclusion and Future Scope

7.1 Conclusion

This project demonstrated a complete, autonomous insurance claims triage system that combines calibrated machine learning with agentic AI orchestration:

1. A **LightGBM model** was trained and calibrated to output true fraud probabilities, achieving 0.874 ROC-AUC and 79% recall on held-out real claims—significantly outperforming logistic regression (0.193 PR-AUC), random forest (0.246), XGBoost (0.283) and a PyTorch MLP (0.262).

2. A **LangGraph-based agent** was built with an 8-step pipeline and 6 specialised tools, autonomously investigating each claim via the ReAct pattern and synthesising evidence into structured, grounded recommendations.

3. **Eight code-enforced guardrails** ensure safety invariants—the system never denies a claim, high-risk claims are never auto-approved, and every assertion traces to its source—independent of LLM behaviour.

4. A **Streamlit dashboard** enables human adjusters to review decisions, examine SHAP explanations, and record approvals or overrides with audit logging.

5. The system achieves **100% consistency** and **zero errors** across repeated evaluation runs, with graceful degradation to a rule-based fallback at zero cost when the LLM is unavailable.

The quantitative results explicitly satisfy the synopsis success criteria: model precision (lower CI 0.188) and recall (lower CI 0.717) are both meaningfully above the naive baseline (0.060 and 0.227 respectively).

7.2 Future Scope

1. **Live LLM evaluation:** run the full agent with qwen2.5:72b on the test set and assess rationale quality via LLM-as-judge.

2. **Feedback loop:** use adjuster override decisions to retrain and recalibrate the model.

3. **Free-text extraction:** parse unstructured claim narratives and repair notes using an LLM.

4. **Multi-line extension:** adapt the pipeline for health, property and life insurance claims.

5. **Real-time deployment:** wrap the tools in a FastAPI service with sub-second latency.

6. **Image-based damage assessment:** integrate computer vision for vehicle damage photos (stretch goal).

7. **Supplementary data sources:** incorporate the Kaggle Auto Insurance Claims dataset ($\sim$1,000 rows) as an additional retrieval knowledge base.

References

[1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: a review," *Statistical Science*, vol. 17, no. 3, pp. 235–249, 2002.

[2] C. Phua, V. Lee, K. Smith and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," *arXiv preprint arXiv:1009.6119*, 2010.

[3] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. ACM SIGKDD*, San Francisco, CA, 2016, pp. 785–794.

[4] G. Ke, Q. Meng, T. Finley *et al.*, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[7] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[8] LangChain, Inc., "LangGraph: build stateful, multi-actor applications with LLMs," *GitHub repository*, 2024. Available: https://github.com/langchain-ai/langgraph.

[9] L. Grinsztajn, E. Oyallon and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 507–520.

[10] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[11] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999, pp. 61–74.

[12] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[13] Streamlit, Inc., "Streamlit: the fastest way to build data apps," 2024. Available: https://streamlit.io.

[14] Ollama, "Ollama: run large language models locally," 2024. Available: https://ollama.ai.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: an imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[17] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: a next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD*, 2019, pp. 2623–2631.

[18] W. Hilal, S. A. Gadsden and J. Yawney, "Financial fraud: a review of anomaly detection techniques and recent advances," *Expert Systems with Applications*, vol. 193, 2022, p. 116429.

[19] B. Baesens, V. Van Vlasselaer and W. Verbeke, *Fraud Analytics Using Descriptive, Predictive, and Social Network Techniques*, Wiley, 2015.

[20] J. West and M. Bhattacharya, "Intelligent financial fraud detection: a comprehensive review," *Computers & Security*, vol. 57, pp. 47–66, 2016.

[21] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. ICML*, 2005, pp. 625–632.

[22] P. Flach and M. Kull, "Precision-recall-gain curves: PR analysis done right," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 838–846.

[23] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[24] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE SSCI*, 2015, pp. 159–166.

[25] LangChain, Inc., "LangChain: building applications with LLMs through composability," *GitHub repository*, 2024. Available: https://github.com/langchain-ai/langchain.

Annexure A: Publications and Intellectual Property

Type	Title / Description	Status
Research Paper	Agentic AI for Insurance Claims Triage	Planned
GitHub Repo	AgenticAIFraudInsuranceDetection	Published

Annexure B: Student Contribution Statement

Student Name	Roll No.	Contribution
Apeksha Sharma	_____	_____
Gunjal Mandawkar	_____	_____
Vaishnavi Singh	_____	_____
Vidhita Bais	_____	_____

Annexure C: Project Outputs

Major outputs generated from the project:

- **Working prototype:** end-to-end fraud triage pipeline (ML model + LangGraph agent + Streamlit dashboard).
- **Trained model:** `lightgbm-b200-20261016-0915` with calibrated probability outputs.
- **Software:** modular Python package (`src/`) with 64 pytest tests.
- **Dataset:** cleaned, split and audited vehicle insurance claims data.
- **Reports:** EDA report, model report, agent evaluation report, model card.
- **GitHub repository:** https://github.com/Vidhita06/AgenticAIFraudInsuranceDetection.
- **Dashboard:** Streamlit application for adjuster review (`dashboard/app.py`).
- **Vector store:** FAISS index over 10,793 training claims for similar-claims retrieval.

Approved by

Guide **Associate Head**

(Name and Signature) (Name and Signature)

Dr. Padma Adane
Dean, SoCSE